

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт цифрового развития

Межинститутская базовая кафедра

КУРСОВОЙ ПРОЕКТ

по дисциплине «Технология разработки программного обеспечения»

на тему:

«Разработка SPA-приложения туристического агентства Travel World»

Выполнил: студент 2 курса, группа ПИЖ-б-о-24-1

направления подготовки 09.03.04 «Программная инженерия»

направленность (профиль) «Разработка и сопровождение программного обеспечения»

очной формы обучения

Руководитель: Самойлов Ф.В., доцент МИБК

Ставрополь, 2026

СОДЕРЖАНИЕ

1. [Аналитическая часть](#)

- 1.1. Описание предметной области
- 1.2. Анализ бизнес-процессов (IDEF0)
- 1.3. SWOT-анализ
- 1.4. Анализ аналогов и конкурентов
- 1.5. Обоснование необходимости разработки

2. [Проектная часть](#)

- 2.1. Модель требований к ПО
- 2.2. Модель предметной области
- 2.3. Архитектурное проектирование
- 2.4. Проектирование данных
- 2.5. Детальное проектирование

3. [Реализационная часть](#)

- 3.1. Структура проекта
- 3.2. Реализация бизнес-логики
- 3.3. Пользовательский интерфейс
- 3.4. Безопасность

4. [Тестирование](#)

5. [Развёртывание](#)

6. [Управление проектом](#)

7. [Заключение](#)

ВВЕДЕНИЕ

Туристическая отрасль является одной из наиболее динамично развивающихся в мире. По данным Всемирной туристской организации ООН, объём мирового туристического рынка ежегодно растёт, при этом значительная доля взаимодействия туристов с агентствами перемещается в цифровую среду. Клиенты ожидают от веб-сайтов туристических агентств мгновенного отклика, интуитивной навигации и современного пользовательского опыта.

Традиционные многостраничные сайты (MPA) уступают место одностраничным приложениям (SPA), которые обеспечивают навигацию без перезагрузки страницы и более высокую производительность. Современные SPA строятся на фреймворках — React, Vue, Angular, — однако для глубокого понимания их внутренней механики ценным опытом является построение собственной архитектуры.

Цель проекта: разработать SPA-приложение туристического агентства «Travel World», реализовав собственную SPA-архитектуру на чистом JavaScript без использования сторонних фреймворков.

Задачи:

1. Провести анализ предметной области туристических веб-сервисов.
2. Разработать модель требований и архитектуру системы.
3. Реализовать кастомный Router с управлением History API.
4. Реализовать систему страниц (Page) с жизненным циклом mount/unmount.
5. Обеспечить CSS-изоляция стилей между страницами.
6. Реализовать клиентскую авторизацию по купону.
7. Подготовить комплект проектной документации.

Объект исследования: архитектура клиентских одностраничных веб-приложений.

Предмет исследования: методы построения SPA без использования JavaScript-фреймворков.

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Описание предметной области

Предметная область проекта — информационная система туристического агентства, предоставляющая пользователям:

- обзор популярных туристических направлений (Токио, Париж, Бали);
- информацию о компании и её деятельности;
- каталог туристических предложений (доступен после авторизации);
- систему авторизации по купону.

Ключевые сущности предметной области:

Сущность	Описание
Туристическое направление	Страна/город с описанием и фотографией
Туристическое предложение	Конкретный тур с ценой, продолжительностью и описанием
Купон	Уникальный код, дающий доступ к предложениям

Страница	Раздел сайта (/home, /about, /offers, /auth)
Сессия	Авторизационное состояние пользователя в браузере

1.2. Анализ бизнес-процессов (IDEF0)

Диаграмма A-0 (контекстная) описывает систему как единый блок «Управление информацией о путешествиях».

Входы (Input):

- Запрос пользователя (URL-переход или клик на ссылку)
- Данные купона (ключевое слово + номер купона)
- Статические данные о турах и направлениях

Управление (Control):

- Бизнес-правила авторизации (купон обязателен для /offers)
- Правила маршрутизации (route guard)
- Политика CSS-изоляции

Механизмы (Mechanism):

- Браузер (среда выполнения)
- History API (навигация)
- localStorage (хранение сессии)
- ES Modules (система модулей)

Выходы (Output):

- Отрендеренная HTML-страница
- Навигационное состояние (History state)
- Авторизованный/гостевой доступ

Декомпозиция A-0 включает пять функциональных блоков: A1 (маршрутизация), A2 (авторизация), A3 (рендеринг страниц), A4 (управление состоянием), A5 (изоляция CSS).

1.3. SWOT-анализ

	Положительные	Отрицательные
Внутренние	Сильные стороны: нет зависимостей от фреймворков; мгновенная загрузка; кастомная архитектура с lifecycle; CSS-изоляция без CSS-in-JS; чистые ES Modules	Слабые стороны: отсутствие backend и реальной БД; данные захардкожены; авторизация только на клиенте; нет поиска и фильтрации
Внешние	Возможности: интеграция Django REST API; расширение до PWA; реальный каталог стран; state manager	Угрозы: рост сложности без фреймворка; браузерная совместимость; 404 при прямом URL; очистка localStorage

1.4. Анализ аналогов и конкурентов

Аналог	Технологии	Преимущества	Недостатки
--------	------------	--------------	------------

Booking.com	React, Node.js, PostgreSQL	Полный функционал, реальные данные, SSR	Сложность, зависимость от инфраструктуры
TripAdvisor	Vue.js, Django	Отзывы, рейтинги, интеграции	Большой объём легаси-кода
Airbnb (frontend)	React, GraphQL	Высокая производительность	Только аренда
Travel World (наш)	Vanilla JS	Нет зависимостей, понятная архитектура	Нет backend

Вывод: существующие решения предоставляют полный функционал, но скрывают механизм работы SPA за фреймворками. Наш проект демонстрирует эти механизмы явно.

1.5. Обоснование необходимости разработки

Разработка Travel World обоснована двумя факторами:

- 1. **Образовательный:** понимание внутреннего устройства SPA-фреймворков (маршрутизация, жизненный цикл, управление состоянием) невозможно без их самостоятельной реализации.
- 2. **Практический:** туристическое агентство нуждается в демонстрационном витрине с быстрой навигацией и системой доступа к специальным предложениям по купону.

2. ПРОЕКТНАЯ ЧАСТЬ

2.1. Модель требований к ПО

Функциональные требования

Публичная часть (все пользователи):

- FR-1: Просмотр главной страницы с направлениями путешествий
- FR-2: Просмотр информации о компании
- FR-3: Навигация между страницами без перезагрузки (SPA)
- FR-4: Авторизация по ключевому слову и купону

Приватная часть (авторизованные пользователи):

- FR-5: Просмотр каталога туристических предложений
- FR-6: Выход из системы (сброс купона)

Нефункциональные требования

Тип	Требование
Производительность	Переход между страницами < 100 мс (без HTTP-запроса)
Совместимость	Chrome 90+, Firefox 88+, Safari 14+
Безопасность	Купон не передаётся по сети; только localStorage
Поддерживаемость	Каждая страница изолирована: стили + логика + HTML

Масштабируемость	Новая страница добавляется регистрацией одной строки в router
------------------	---

Use Case сводка

Use Case	Актор	Требует авторизации
UC1: Главная страница	Гость, Авторизованный	Нет
UC2: Страница «О нас»	Гость, Авторизованный	Нет
UC3: Авторизация	Гость	Нет
UC4: Просмотр предложений	Авторизованный	Да
UC5: SPA-навигация	Все	Нет
UC6: Выход	Авторизованный	Да

Глоссарий терминов

Термин	Определение
SPA	Single Page Application — приложение в одной HTML-странице без перезагрузки
Router	Модуль навигации через History API
Page	Абстракция страницы: HTML + CSS + mount/unmount
Mount	Инициализация страницы: вставка HTML, подключение стилей, навешивание событий
Unmount	Очистка страницы: удаление обработчиков и стилей
Navigate	Функция перехода без перезагрузки браузера
Credentials	Объект авторизации {keyword, coupon, authorizedAt} в localStorage
Route Guard	Проверка авторизации при доступе к защищённому маршруту
CSS Isolation	Стратегия подключения/удаления CSS-файлов при смене страниц
History API	Браузерный API для управления URL без перезагрузки (pushState/popstate)
LocalStorage	Браузерное хранилище ключ-значение для персистентных данных
ES Modules	Стандарт JavaScript-модулей (import/export)

2.2. Модель предметной области

Диаграмма классов

Система состоит из следующих ключевых классов:

- **Router** — управляет маршрутами и рендерингом страниц
- **Page** — абстракция страницы (html, title, mountFn)
- **CSSManager** — динамическое управление стилями
- **NavigateHelper** — функция перехода между страницами

- **AuthService** (неявный) — работа с localStorage

Бизнес-правила:

- BR-1: Доступ к /offers — только авторизованным
- BR-2: Route Guard перенаправляет на /auth при отсутствии credentials
- BR-3: CSS предыдущей страницы удаляется при переходе
- BR-4: Каждая страница возвращает cleanup() из mount()
- BR-5: Навигация только через navigate(), не через href

2.3. Архитектурное проектирование

Архитектурный стиль

Приложение реализовано по паттерну **компонентной архитектуры** (Component-Based Architecture) с кастомным слоем маршрутизации, аналогичным React Router.

Вместо фреймворка реализованы собственные абстракции:

Абстракция	Аналог в React/Vue
Router	React Router
Page	Компонент (Component)
mount/unmount	componentDidMount/componentWillUnmount
CSSManager	CSS Modules
navigate()	useNavigate()

Диаграмма компонентов

```
index.html → main.js → Router
                        ├── homePage   (/home)
                        ├── aboutPage  (/about)
                        ├── offersPage (/offers) [guard]
                        └── authPage   (/auth)

shared/
├── utils/router.js    – маршрутизация
├── utils/page.js      – фабрика страниц
├── helpers/navigate.js – навигация
└── helpers/pageStyles.js – CSS-изоляция
```

Схема взаимодействия клиент-браузер

Приложение полностью клиентское. Сервер отдаёт только `index.html` и статические файлы. Вся логика работает в браузере:

```
Пользователь → Клик/URL → Router.render() → Page.mount() → DOM обновлён
```

2.4. Проектирование данных

Логическая модель данных

Данные хранятся в трёх местах:

- 1. **localStorage** — авторизационная сессия (credentials)
- 2. **JavaScript-код** — статический контент (направления, предложения)
- 3. **public/images/** — медиафайлы (изображения мест)

Структура credentials (localStorage)

Поле	Тип	Ограничение
keyword	string	NOT NULL
coupon	string	NOT NULL
authorizedAt	number	NOT NULL (Unix ms)

ER-диаграмма (концептуальная)

Концептуальная ER-модель включает сущности: Credentials , TravelDestination , TravelOffer , Route . Подробное описание приведено в [docs/08-er-diagrams/er-diagram.md](#).

Нормализация

Все структуры данных приведены к 3-й нормальной форме: отсутствуют повторяющиеся группы (1НФ), частичные зависимости (2НФ) и транзитивные зависимости (3НФ).

2.5. Детальное проектирование

Диаграмма последовательности: авторизация

```
Гость → /auth → AuthPage.mount()
Гость → вводит keyword + coupon
AuthPage → validates(form)
AuthPage → localStorage.setItem('credentials', JSON)
AuthPage → navigate('/offers')
Router → offersPage.mount()
```

Диаграмма последовательности: SPA-навигация

```
Пользователь → click(link)
link → preventDefault()
link → navigate('/target')
navigate → window.dispatchEvent(routeChange)
Router → handle(routeChange)
Router → currentCleanup() ← unmount старой страницы
Router → page.mount(doc) ← mount новой страницы
Router → history.pushState(url)
```

Применение паттернов

Паттерн	Применение
Factory	Page(path, title, html, mountFn) — фабрика страниц
Observer	CustomEvent('routeChange') + addEventListener — Router слушает навигацию

Strategy	CSSManager меняет стратегию стилей при переходе
Template Method	mount() определяет шаблон жизненного цикла
Guard Clause	Route guard в Router.render() перед монтированием

3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ

3.1. Структура проекта

```
sophia-1-travel-app/
├─ index.html           # Точка входа SPA
├─ src/
│  ├─ main.js           # Регистрация маршрутов
│  ├─ pages/
│  │  ├─ home.js        # Главная страница
│  │  ├─ about.js       # О компании
│  │  ├─ offers.js       # Туристические предложения
│  │  └─ auth.js        # Авторизация по купону
│  ├─ shared/
│  │  ├─ utils/
│  │  │  ├─ router.js   # Кастомный SPA-роутер
│  │  │  └─ page.js     # Фабрика страниц
│  │  ├─ helpers/
│  │  │  ├─ navigate.js # Функция навигации
│  │  │  └─ pageStyles.js # CSS-менеджер
│  │  └─ consts/
│  │     └─ events.js   # Константы событий
│  └─ assets/           # SVG-ресурсы
├─ public/
│  ├─ styles/           # CSS-файлы страниц
│  │  ├─ home.css
│  │  ├─ about.css
│  │  ├─ offers.css
│  │  └─ auth.css
│  ├─ images/           # Локальные изображения
│  │  ├─ hero-beach.jpg # Фон главной страницы
│  │  ├─ tokyo.jpg
│  │  ├─ paris.jpg
│  │  └─ bali.jpg
│  ├─ favicon.svg
│  └─ icons.svg
└─ docs/                # Документация
```

3.2. Реализация бизнес-логики

3.2.1. Кастомный Router (`src/shared/utils/router.js`)

Router реализует следующую логику:

1. Хранит Map маршрутов `path → Page`

2. При вызове `render(pathname)` — вызывает `cleanup` предыдущей страницы, монтирует новую
3. Подписывается на `popstate` (кнопка «Назад») и кастомное событие `routeChange`
4. Реализует `route guard`: проверяет наличие `credentials` перед монтированием `/offers`

3.2.2. Фабрика Page (`src/shared/utils/page.js`)

Функция `Page(path, title, html, mountFn)` возвращает объект с методами:

- `getPath()` — возвращает URL маршрута
- `getTitle()` — возвращает заголовок страницы
- `mount(document)` — вставляет HTML в DOM, подключает CSS, вызывает `mountFn`, возвращает `cleanup`

3.2.3. CSS-изоляция (`src/shared/helpers/pageStyles.js`)

При каждом монтировании страницы:

1. Удаляется предыдущий `<link>` со стилями
2. Создаётся новый `<link href="/styles/{page}.css">` и добавляется в `<head>`

Это предотвращает конфликты CSS между страницами без CSS Modules или Shadow DOM.

3.2.4. Авторизация (`src/pages/auth.js`)

Авторизация реализована через `localStorage`:

```
{
  keyword: "TRAVEL2026",
  coupon: "TW-45821",
  authorizedAt: Date.now()
}
```

После успешного ввода данные сохраняются, и `navigate('/offers')` перенаправляет пользователя.

3.3. Пользовательский интерфейс

Страницы приложения

Страница	URL	Описание
Главная	<code>/home</code>	Навигация, hero-баннер, 3 карточки направлений
О нас	<code>/about</code>	Информация о компании Travel World
Предложения	<code>/offers</code>	Каталог туров (только авторизованным)
Авторизация	<code>/auth</code>	Форма ввода купона

Особенности UI

- Адаптивная сетка карточек направлений (`grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))`)
- Hero-баннер с изображением пляжа в полную высоту экрана
- Плавные переходы цвета кнопок и ссылок (`transition: 0.3s`)
- Все изображения хранятся локально (без CDN-зависимостей)

3.4. Безопасность и ограничения

Аспект	Реализация
Защита маршрута /offers	Route guard в Router перед монтированием страницы
Хранение купона	localStorage — только в браузере пользователя, не передаётся по сети
XSS	Статический HTML, нет динамического innerHTML с пользовательским вводом
Ограничение	Авторизация не является серверной — является демонстрационной

4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА

4.1. Тест-кейсы навигации

№	Тест	Действие	Ожидаемый результат	Статус
T-01	Загрузка приложения	Открыть / в браузере	Перенаправление на /home, главная страница загружена	✓ Pass
T-02	Навигация: /about	Клик «О нас»	Страница /about загружена без перезагрузки, URL обновлён	✓ Pass
T-03	Кнопка «Назад»	navigate('/about') → нажать «Назад»	Возврат на /home через popstate	✓ Pass
T-04	CSS изоляция	Переключить home → about	Стили home.css удалены, about.css подключён	✓ Pass

4.2. Тест-кейсы авторизации

№	Тест	Действие	Ожидаемый результат	Статус
T-05	Доступ к /offers без купона	Перейти на /offers	Route Guard → редирект на /auth	✓ Pass
T-06	Авторизация с купоном	Ввести keyword + coupon → Войти	credentials в localStorage, редирект на /offers	✓ Pass
T-07	Доступ после авторизации	Открыть /offers после T-06	Страница предложений загружена	✓ Pass
T-08	Сохранение сессии	Перезагрузить страницу после T-06	Авторизация сохранена (localStorage)	✓ Pass

4.3. Системное тестирование

Тестирование проводилось в следующих браузерах:

Браузер	Версия	Результат
---------	--------	-----------

Google Chrome	124+	✔ Работает корректно
Mozilla Firefox	125+	✔ Работает корректно
Safari	17+	✔ Работает корректно

5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

5.1. Требования к окружению

Компонент	Версия
Node.js	18+
npm	9+
Vite (dev server)	5+
Браузер	Chrome 90+ / Firefox 88+ / Safari 14+

5.2. Инструкция по установке (локальный запуск)

```
# 1. Клонировать репозиторий
git clone <repo-url>
cd sophia-1-travel-app

# 2. Установить зависимости
npm install

# 3. Запустить dev-сервер
npm run dev

# 4. Открыть в браузере
# http://localhost:5173
```

5.3. Настройка Vite

Важно: при прямом доступе по URL (например `/offers`) без клиентского роутера браузер вернёт 404. Vite автоматически решает это через настройку `historyApiFallback` , перенаправляя все запросы на `index.html` .

5.4. Структура статических файлов

Все изображения хранятся локально в `public/images/` и не требуют подключения к внешним CDN-сервисам:

Файл	Назначение
hero-beach.jpg	Фоновое изображение hero-секции главной страницы
tokyo.jpg	Карточка направления «Токио»

paris.jpg	Карточка направления «Париж»
bali.jpg	Карточка направления «Бали»

6. УПРАВЛЕНИЕ ПРОЕКТОМ

6.1. WBS (Иерархическая структура работ)

Полная WBS приведена в <docs/09-wbs-gantt/wbs-gantt.md>.

Основные группы работ:

1. Инициация и анализ (8 ч)
2. Проектирование требований (12 ч)
3. Архитектурное проектирование (8 ч)
4. Проектирование данных (4 ч)
5. Реализация (24 ч)
6. Тестирование (6 ч)
7. Документирование (16 ч)

Итого: ~78 часов

6.2. Диаграмма Ганта

Диаграмма Ганта приведена в <docs/09-wbs-gantt/wbs-gantt.md>.

Контрольные точки:

- 12 марта — 25% (аналитическая часть)
- 09 апреля — 50% (проектная часть)
- 30 апреля — 75% (реализация)
- 14 мая — 100% (документация, сдача)
- 23 мая — защита

6.3. Оценка трудозатрат (COCOMO)

Объём кода проекта составляет около 450 строк JS + 400 строк CSS. По модели COCOMO (organic mode) для малых проектов:

- Расчётное время разработки: 2–3 недели
- Фактическое: 10 недель (с учётом документирования)

6.4. Управление рисками

Подробная таблица рисков приведена в <docs/09-wbs-gantt/wbs-gantt.md>.

Ключевые риски:

- R2 (высокое влияние): LocalStorage заблокирован в режиме инкогнито → обработка ошибок записи
- R4 (среднее влияние): Прямой URL без роутера → Vite fallback на index.html

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработано SPA-приложение туристического агентства «Travel World» с собственной frontend-архитектурой на чистом JavaScript без использования сторонних фреймворков.

Достигнутые результаты:

1. Проведён анализ предметной области и построены диаграммы IDEF0, BUC, Use Case.
2. Разработана кастомная SPA-архитектура с роутером, системой страниц и управлением жизненным циклом.
3. Реализованы четыре страницы приложения (/home, /about, /offers, /auth) с изолированными стилями.
4. Реализована клиентская авторизация по купону с route guard.
5. Подготовлен полный комплект проектной документации.

Практическая ценность: проект демонстрирует, как устроены современные SPA-фреймворки «под капотом» — навигация через History API, жизненный цикл компонентов, изоляция стилей. Эти знания являются фундаментом для перехода к React, Vue или Angular.

Перспективы развития:

- Интеграция Django REST API (переход к траектории Б)
- Реальный каталог туров из базы данных
- PWA (Service Worker, offline-режим)
- Lazy loading страниц
- Кастомный state manager (mini-store)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кузьмин, А. Г. Разработка фронтенд-приложений / А. Г. Кузьмин. — Санкт-Петербург : Питер, 2025. — 496 с. — ISBN 978-5-4461-4272-9.
2. Кухтин, С. А. Frontend-разработка сайтов и приложений на HTML, CSS, JavaScript и React / С. А. Кухтин. — Санкт-Петербург : Наука и Техника, 2026. — 512 с. — ISBN 978-5-907592-84-1.
3. Орбайсета, А. С. Создание фронтенд-фреймворка с нуля / А. С. Орбайсета. — Санкт-Петербург : Питер, 2025. — 384 с. — ISBN 978-5-4461-4201-9.
4. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и node.js : учебное пособие для вузов. — 4-е изд., стер. — Санкт-Петербург : Лань, 2025. — 120 с. — ISBN 978-5-507-51011-5.
5. Mozilla Developer Network. History API [Электронный ресурс]. — URL: https://developer.mozilla.org/ru/docs/Web/API/History_API (дата обращения: 15.03.2026).
6. Mozilla Developer Network. Window: localStorage property [Электронный ресурс]. — URL: <https://developer.mozilla.org/ru/docs/Web/API/Window/localStorage> (дата обращения: 15.03.2026).
7. Mozilla Developer Network. ES Modules [Электронный ресурс]. — URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript/Guide/Modules> (дата обращения: 16.03.2026).
8. Mozilla Developer Network. CustomEvent [Электронный ресурс]. — URL: <https://developer.mozilla.org/ru/docs/Web/API/CustomEvent> (дата обращения: 16.03.2026).
9. Vite. Official Documentation [Электронный ресурс]. — URL: <https://vitejs.dev/guide> (дата обращения: 20.03.2026).

10. IDEF0 Function Modeling Method. Federal Information Processing Standards Publication 183. — Washington : NIST, 1993.
 11. Буч, Г. Объектно-ориентированный анализ и проектирование с примерами приложений / Г. Буч, Р. Максимчук, М. Энгл. — Москва : Вильямс, 2008. — 720 с.
 12. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. — Москва : Вильямс, 2012. — 544 с.
 13. Гамма, Э. Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. — Санкт-Петербург : Питер, 2015. — 366 с.
 14. Симпсон, К. Вы не знаете JavaScript: замыкания и объекты / К. Симпсон. — Москва : Питер, 2018.
 15. Флэнаган, Д. JavaScript. Подробное руководство / Д. Флэнаган. — 7-е изд. — Санкт-Петербург : БХВ-Петербург, 2021. — 706 с.
 16. Яндекс Практикум. Курс «Фронтенд-разработчик» [Электронный ресурс]. — URL: <https://practicum.yandex.ru/frontend-developer> (дата обращения: 10.02.2026).
 17. Metanit.com. JavaScript. Руководство [Электронный ресурс]. — URL: <https://metanit.com/web/javascript> (дата обращения: 12.02.2026).
 18. Чернышев, С. А. Основы Flutter / С. А. Чернышев, Ю. М. Петров. — Санкт-Петербург : Питер, 2026. — 688 с.
 19. W3C. Web Storage Specification [Электронный ресурс]. — URL: <https://www.w3.org/TR/webstorage> (дата обращения: 18.03.2026).
 20. W3C. HTML Living Standard — History interface [Электронный ресурс]. — URL: <https://html.spec.whatwg.org/multipage/history.html> (дата обращения: 18.03.2026).
 21. ГОСТ 2.105-95. Единая система конструкторской документации. Общие требования к текстовым документам. — Москва : Стандартинформ, 2007.
 22. ГОСТ Р 7.0.11-2011. Система стандартов по информации. Диссертация и автореферат диссертации. — Москва : Стандартинформ, 2012.
 23. ГОСТ 7.32-2001. Отчёт о научно-исследовательской работе. Структура и правила оформления. — Москва : ИПК Издательство стандартов, 2002.
 24. Nielsen, J. Usability Engineering / J. Nielsen. — Morgan Kaufmann, 1994.
 25. Всемирная туристская организация ООН. Барометр туризма [Электронный ресурс]. — URL: <https://www.unwto.org/tourism-statistics> (дата обращения: 05.02.2026).
-

ПРИЛОЖЕНИЯ

Приложение А. Листинги ключевых модулей

Ключевые файлы проекта расположены в:

- [src/shared/utils/router.js](#) — Router
- [src/shared/utils/page.js](#) — Page factory
- [src/shared/helpers/navigate.js](#) — навигация
- [src/shared/helpers/pageStyles.js](#) — CSS-менеджер

- [src/pages/home.js](#) — главная страница
- [src/pages/auth.js](#) — авторизация

Приложение Б. Скриншоты интерфейсов

Скриншоты приложения расположены в корне репозитория:

- [app1.png](#)
- [app2.png](#)
- [app3.png](#)
- [app4.png](#)

Приложение В. Результаты тестирования

Результаты тестирования приведены в разделе 4 данного документа.